

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Nazmi Hakim NIM. 2310817210012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nazmi Hakim
NIM : 2310817210012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	7
B. Output Program	23
C. Pembahasan	31
D. Tautan Git	34
SOAL 2.....	35
A. Pembahasan	35

DAFTAR GAMBAR

Gambar 1. Contoh Penggunaan Debugger	6
Gambar 2. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	23
Gambar 3. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	23
Gambar 4. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	24
Gambar 5. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	24
Gambar 6. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	25
Gambar 7. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	25
Gambar 8. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	26
Gambar 9. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	26
Gambar 10. Screenshot Hasil Jawaban Soal 1 XML.....	27
Gambar 11. Screenshot Hasil Jawaban Soal 1 XML.....	27
Gambar 12. Screenshot Hasil Jawaban Soal 1 XML.....	28
Gambar 13. Screenshot Hasil Jawaban Soal 1 XML.....	28
Gambar 14. Screenshot Hasil Jawaban Soal 1 XML.....	29
Gambar 15. Screenshot Hasil Jawaban Soal 1 XML.....	29
Gambar 16. Screenshot Hasil Jawaban Soal 1 XML.....	30
Gambar 17. Screenshot Hasil Jawaban Soal 1 XML.....	30
Gambar 18. Screenshot Hasil Jawaban Soal 1 XML.....	31

DAFTAR TABEL

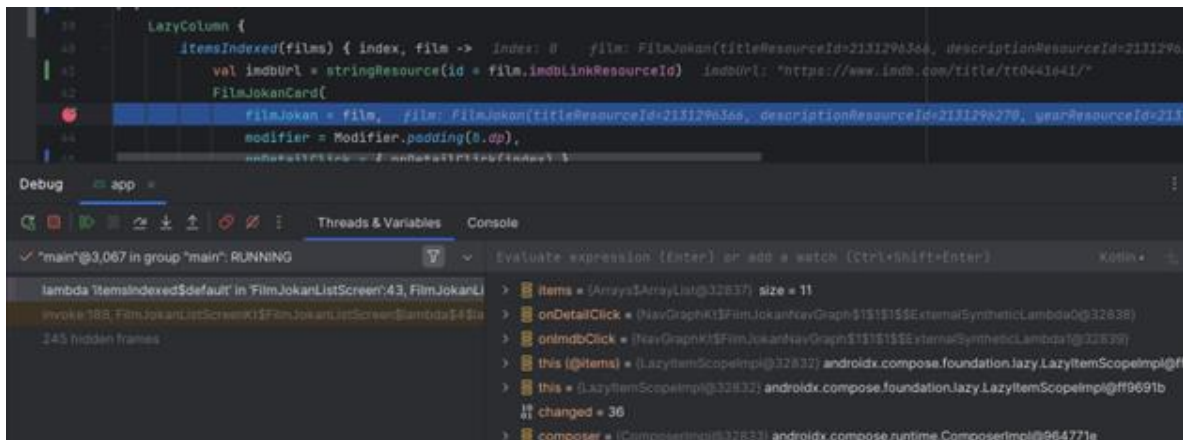
Tabel 1. Source Code Jawaban Soal 1 Jetpack Compose.....	7
Tabel 2. Source Code Jawaban Soal 1 Jetpack Compose.....	8
Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose.....	9
Tabel 4. Source Code Jawaban Soal 1 Jetpack Compose.....	11
Tabel 5. Source Code Jawaban Soal 1 Jetpack Compose.....	12
Tabel 6. Source Code Jawaban Soal 1 Jetpack Compose.....	12
Tabel 7. Source Code Jawaban Soal 1 XML.....	13
Tabel 8. Source Code Jawaban Soal 1 XML.....	13
Tabel 9. Source Code Jawaban Soal 1 XML.....	14
Tabel 10. Source Code Jawaban Soal 1 XML.....	15
Tabel 11. Source Code Jawaban Soal 1 XML.....	17
Tabel 12. Source Code Jawaban Soal 1 XML.....	18
Tabel 13. Source Code Jawaban Soal 1 XML.....	18
Tabel 14. Source Code Jawaban Soal 1 XML.....	19
Tabel 15. Source Code Jawaban Soal 1 XML.....	20
Tabel 16. Source Code Jawaban Soal 1 XML.....	21
Tabel 17. Source Code Jawaban Soal 1 XML.....	21
Tabel 18. Source Code Jawaban Soal 1 XML.....	22

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- Install dan gunakan library Timber untuk logging event berikut:
 - Log saat data item masuk ke dalam list
 - Log saat tombol Detail dan tombol Explicit Intent ditekan
 - Log data dari list yang dipilih ketika berpindah ke halaman Detail
- Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 1. Contoh Penggunaan Debugger

A. Source Code

Jetpack Compose :

MainActivity.kt :

Tabel 1. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.navigation.NavType
7	import androidx.navigation.compose.NavHost
8	import androidx.navigation.compose.composable
9	import androidx.navigation.compose.rememberNavController
10	import androidx.navigation.navArgument
11	import com.example.listcompose.ui.theme.ListComposeTheme
12	import timber.log.Timber
13	
14	class MainActivity : ComponentActivity() {
15	override fun onCreate(savedInstanceState: Bundle?) {
16	super.onCreate(savedInstanceState)
17	Timber.plant(Timber.DebugTree())
18	setContent {
19	ListComposeTheme {
20	val navController = rememberNavController()
21	NavHost(navController, startDestination =
22	"list") {
23	composable("list") {
24	ItemListScreen(navController)
25	}
26	composable(
27	"detail/{itemId}",
28	arguments =
29	listOf(navArgument("itemId") { type = NavType.IntType })
30	) { backStackEntry ->
31	val itemId =
32	backStackEntry.arguments?.getInt("itemId") ?: 0
33	val context = this@MainActivity
34	val items = getItems(context)
35	DetailScreen(item = items[itemId])
36	}
37	}
38	}
	}
	}

Item.kt :

Tabel 2. Source Code Jawaban Soal 1 Jetpack Compose

```
1 package com.example.listcompose
2
3 import android.content.Context
4
5 data class Item(
6     val id: Int,
7     val title: String,
8     val author: String,
9     val info1: String,
10    val info2: String,
11    val imageResId: Int,
12    val url: String,
13    val description: String
14 )
15
16 fun.getItems(context: Context): List<Item> {
17     val titles =
18 context.resources.getStringArray(R.array.item_titles)
19     val authors =
20 context.resources.getStringArray(R.array.item_authors)
21     val infos1 =
22 context.resources.getStringArray(R.array.item_info1)
23     val infos2 =
24 context.resources.getStringArray(R.array.item_info2)
25     val urls =
26 context.resources.getStringArray(R.array.item_urls)
27     val descriptions =
28 context.resources.getStringArray(R.array.item_descriptions)
29
30     val imageIds = listOf(
31         R.drawable.lookism,
32         R.drawable.bones,
33         R.drawable.wee,
34         R.drawable.herohasreturned,
35         R.drawable.studygroup
36     )
37
38     return titles.indices.map { index ->
39         Item(
40             id = index,
41             title = titles[index],
42             author = authors[index],
43             info1 = infos1[index],
44             info2 = infos2[index],
45             imageResId = imageIds[index],
46             url = urls[index],
47             description = descriptions[index]
48         )
49     }
```

	<pre> } } </pre>
--	------------------------------

ItemListScreen.kt :

Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import android.content.Intent
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.Arrangement
6	import androidx.compose.foundation.layout.Column
7	import androidx.compose.foundation.layout.PaddingValues
8	import androidx.compose.foundation.layout.Row
9	import androidx.compose.foundation.layout.Spacer
10	import androidx.compose.foundation.layout.fillMaxWidth
11	import androidx.compose.foundation.layout.height
12	import androidx.compose.foundation.layout.padding
13	import androidx.compose.foundation.lazy.LazyColumn
14	import androidx.compose.foundation.shape.RoundedCornerShape
15	import androidx.compose.material3.Button
16	import androidx.compose.material3.Card
17	import androidx.compose.material3.CardDefaults
18	import androidx.compose.material3.MaterialTheme
19	import androidx.compose.material3.Text
20	import androidx.compose.runtime.Composable
21	import androidx.compose.runtime.collectAsState
22	import androidx.compose.runtime.getValue
23	import androidx.compose.ui.Modifier
24	import androidx.compose.ui.draw.clip
25	import androidx.compose.ui.graphics.Color
26	import androidx.compose.ui.layout.ContentScale
27	import androidx.compose.ui.platform.LocalContext
28	import androidx.compose.ui.res.painterResource
29	import androidx.compose.ui.res.stringResource
30	import androidx.compose.ui.unit.dp
31	import androidx.core.net.toUri
32	import androidx.navigation.NavController
33	import timber.log.Timber
34	
35	@Composable
36	fun ItemListScreen(navController: NavController, viewModel:
37	ItemViewModel =
38	androidx.lifecycle.viewmodel.compose.viewModel(factory =
39	ItemViewModelFactory(LocalContext.current, "List Loaded")) {
40	val context = LocalContext.current
41	val itemList by viewModel.itemList.collectAsState()
42	
43	LazyColumn(contentPadding = PaddingValues(8.dp)) {

```

44         items(itemList.size) { index ->
45             val item = itemList[index]
46             Card(
47                 shape = RoundedCornerShape(16.dp),
48                 modifier = Modifier
49                     .padding(8.dp)
50                     .fillMaxWidth(),
51                 colors = CardDefaults.cardColors(containerColor
52 = Color.DarkGray),
53                 elevation =
54 CardDefaults.cardElevation(defaultElevation = 4.dp)
55             ) {
56                 Column(modifier = Modifier.padding(16.dp)) {
57                     Image(
58                         painter = painterResource(id =
59 item.imageResId),
60                         contentDescription = item.title,
61                         modifier = Modifier
62                             .fillMaxWidth()
63                             .height(180.dp)
64                             .clip(RoundedCornerShape(16.dp)),
65                         contentScale = ContentScale.Crop
66                     )
67                     Spacer(modifier = Modifier.height(8.dp))
68                     Text(text = item.title, style =
69 MaterialTheme.typography.titleLarge, color = Color.White)
70                     Text(text = item.author, style =
71 MaterialTheme.typography.bodyMedium, color = Color.White)
72                     Text(text = item.info1, style =
73 MaterialTheme.typography.bodySmall, color = Color.White)
74                     Text(text = item.info2, style =
75 MaterialTheme.typography.bodySmall, color = Color.White)
76                     Spacer(modifier = Modifier.height(8.dp))
77                     Row(horizontalArrangement =
78 Arrangement.spacedBy(8.dp)) {
79                         Button(onClick = {
80                             val intent =
81 Intent(Intent.ACTION_VIEW, item.url.toUri())
82                             Timber.d("Explicit intent to URL:
83 ${item.url}")
84                             context.startActivity(intent)
85                         }) {
86                             Text(text =
stringResource(R.string.manhwa_website))
87                         }
88                         Button(onClick = {
89                             navController.navigate("detail/${item.id}")
90                             Timber.d("Navigated to Detail for
item id: ${item.id}")
91                         }) {
92                             Text(text =

```

	<pre> stringResource(R.string.manhwa_detail)) } } } } } } } } } </pre>
--	--

DetailScreen.kt :

Tabel 4. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import androidx.compose.foundation.verticalScroll
7	import androidx.compose.foundation.shape.RoundedCornerShape
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.draw.clip
12	import androidx.compose.ui.layout.ContentScale
13	import androidx.compose.ui.res.painterResource
14	import androidx.compose.ui.unit.dp
15	
16	@Composable
17	fun DetailScreen(item: Item) {
18	val scrollState = rememberScrollState()
19	Column(
20	modifier = Modifier
21	.padding(16.dp)
22	.fillMaxSize()
23	.verticalScroll(scrollState)
24	) {
25	Image(
26	painter = painterResource(id = item.imageResId),
27	contentDescription = item.title,
28	modifier = Modifier
29	.fillMaxWidth()
30	.height(240.dp)
31	.clip(RoundedCornerShape(16.dp)),
32	contentScale = ContentScale.Crop
33	)
34	Spacer(modifier = Modifier.height(16.dp))
35	Text(text = item.title, style =
36	MaterialTheme.typography.headlineMedium)
37	Text(text = item.author, style =

38	MaterialTheme.typography.bodyLarge)
39	Text(text = item.info1, style =
40	MaterialTheme.typography.bodyMedium)
41	Text(text = item.info2, style =
	MaterialTheme.typography.bodyMedium)
	Text(text = item.description, style =
	MaterialTheme.typography.bodyMedium)
	}
	}

ItemViewModel.kt :

Tabel 5. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import android.content.Context
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.viewModelScope
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import kotlinx.coroutines.launch
9	import timber.log.Timber
10	
11	class ItemViewModel(private val context: Context, private val
12	param: String) : ViewModel() {
13	
14	private val _itemList =
15	MutableStateFlow<List<Item>>(emptyList())
16	val itemList: StateFlow<List<Item>> = _itemList
17	
18	init {
19	viewModelScope.launch {
20	val items =.getItems(context)
21	_itemList.value = items
22	Timber.d("ItemViewModel: \$param - Loaded
23	\${items.size} items")
	}
	}
	}

ItemViewModelFactory.kt :

Tabel 6. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import android.content.Context
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelProvider

6	
7	class ItemViewModelFactory(private val context: Context,
8	private val param: String) : ViewModelProvider.Factory {
9	override fun <T : ViewModel> create(modelClass: Class<T>):
10	T {
11	return ItemViewModel(context, param) as T
	}
	}

XML :

Item.kt :

Tabel 7. Source Code Jawaban Soal 1 XML

1	package	com.example.listxml
2		
3	data	class Item(
4	val	imageResId: Int,
5	val	title: String,
6	val	author: String,
7	val url: String)	

ItemAdapter.kt :

Tabel 8. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import android.annotation.SuppressLint
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.recyclerview.widget.RecyclerView
7	import com.example.listxml.databinding.ItemListBinding
8	
9	class ItemAdapter(
10	private var items: List<Item>,
11	private val onClick: (String) -> Unit,
12	private val onDetailClick: (Item) -> Unit
13	) : RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() {
14	
15	inner class ItemViewHolder(val binding: ItemListBinding) :
16	RecyclerView.ViewHolder(binding.root)
17	
18	override fun onCreateViewHolder(parent: ViewGroup,
19	viewType: Int): ItemViewHolder {
20	val binding =
21	ItemListBinding.inflate(LayoutInflater.from(parent.context),
22	parent, false)
23	return ItemViewHolder(binding)
24	}

25	
26	override fun onBindViewHolder(holder: ItemViewHolder,
27	position: Int) {
28	val item = items[position]
29	with(holder.binding) {
30	itemImage.setImageResource(item.imageResId)
31	titleText.text = item.title
32	subtitleText.text = item.author
33	
34	openButton.setOnClickListener {
35	onOpenClick(item.url)
36	}
37	detailButton.setOnClickListener {
38	onDetailClick(item)
39	}
40	}
41	}
42	
43	override fun getItemCount(): Int = items.size
44	@SuppressWarnings("NotifyDataSetChanged")
	fun updateItems(newItems: List<Item>) {
	items = newItems
	notifyDataSetChanged()
	}
	}

item_list.xml :

Tabel 9. Source Code Jawaban Soal 1 XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<androidx.cardview.widget.CardView		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:card_view="http://schemas.android.com/apk/res-auto"		
5	xmlns:app="http://schemas.android.com/apk/res-auto"		
6	android:layout_width="match_parent"		
7	android:layout_height="wrap_content"		
8	android:layout_marginBottom="12dp"		
9	card_view:cardCornerRadius="16dp"		
10	card_view:cardElevation="4dp">		
11			
12	<LinearLayout		
13	android:layout_width="match_parent"		
14	android:layout_height="wrap_content"		
15	android:orientation="vertical"		
16	android:padding="16dp">		
17			
18			
19	<com.google.android.material.imageview.ShapeableImageView		
20	android:id="@+id/itemImage"		

21	android:layout_width="match_parent"
22	android:layout_height="180dp"
23	android:scaleType="centerCrop"
24	
25	app:shapeAppearanceOverlay="@style/RoundedImageView"/>
26	
27	<TextView
28	android:id="@+id/titleText"
29	android:layout_width="wrap_content"
30	android:layout_height="wrap_content"
31	android:text="Judul"
32	android:textStyle="bold"
33	android:textSize="18sp"
34	android:paddingTop="8dp"/>
35	
36	<TextView
37	android:id="@+id/subtitleText"
38	android:layout_width="wrap_content"
39	android:layout_height="wrap_content"
40	android:text="Subjudul" />
41	
42	<LinearLayout
43	android:layout_width="match_parent"
44	android:layout_height="wrap_content"
45	android:orientation="horizontal"
46	android:gravity="end">
47	
48	<Button
49	android:id="@+id/openButton"
50	android:layout_width="wrap_content"
51	android:layout_height="wrap_content"
52	android:text="Open" />
53	
54	<Button
55	android:id="@+id/detailButton"
56	android:layout_width="wrap_content"
57	android:layout_height="wrap_content"
58	android:text="Detail"
59	android:layout_marginStart="8dp"/>
60	</LinearLayout>
	</LinearLayout>
	</androidx.cardview.widget.CardView>

HomeFragment.kt :

Tabel 10. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import android.content.Intent

```

4 import android.os.Bundle
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import androidx.core.net.toUri
9 import androidx.fragment.app.Fragment
10 import androidx.fragment.app.viewModels
11 import androidx.lifecycle.lifecycleScope
12 import androidx.navigation.fragment.findNavController
13 import androidx.recyclerview.widget.LinearLayoutManager
14 import com.example.listxml.databinding.FragmentHomeBinding
15 import kotlinx.coroutines.launch
16 import timber.log.Timber
17
18 class HomeFragment : Fragment() {
19     private var _binding: FragmentHomeBinding? = null
20     private val binding get() = _binding!!
21
22     private lateinit var adapter: ItemAdapter
23
24     private val viewModel: ItemViewModel by viewModels {
25         ItemViewModelFactory("ParamDariHome")
26     }
27
28     override fun onCreateView(
29         inflater: LayoutInflater, container: ViewGroup?,
30         savedInstanceState: Bundle?
31     ): View {
32         _binding = FragmentHomeBinding.inflate(inflater,
33 container, false)
34         return binding.root
35     }
36
37     override fun onViewCreated(view: View, savedInstanceState:
38 Bundle?) {
39         adapter = ItemAdapter(
40             emptyList(),
41             onOpenClick = { url ->
42                 Timber.d("HomeFragment: Open URL $url")
43                 val intent = Intent(Intent.ACTION_VIEW,
44 url.toUri())
45                 startActivity(intent)
46             },
47             onDetailClick = { item ->
48                 viewModel.selectItem(item)
49                 Timber.d("HomeFragment: Navigate to detail
50 ${item.title}")
51                 val action =
52 HomeFragmentDirections.actionHomeToDetail(
53                     item.title,
54                     item.author,
55                     item.imageResId

```

56	)
57	findNavController().navigate(action)
58	}
59	)
60	
61	binding.recyclerView.layoutManager =
62	LinearLayoutManager(requireContext())
63	binding.recyclerView.adapter = adapter
64	
65	viewLifecycleOwner.lifecycleScope.launch {
66	viewModel.itemList.collect { items ->
67	adapter.updateItems(items)
68	Timber.d("HomeFragment: Received \${items.size}
69	items")
70	}
71	}
	}
	override fun onDestroyView() {
	super.onDestroyView()
	_binding = null
	}
	}

fragment_home.xml :

Tabel 11. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/android"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent">
8	
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/recyclerView"
11	android:layout_width="0dp"
12	android:layout_height="0dp"
13	android:clipToPadding="false"
14	android:padding="16dp"
15	app:layoutManager=
16	"androidx.recyclerview.widget.LinearLayoutManager"
17	app:layout_constraintTop_toTopOf="parent"
18	app:layout_constraintBottom_toBottomOf="parent"
19	app:layout_constraintStart_toStartOf="parent"
	app:layout_constraintEnd_toEndOf="parent"/>
	</androidx.constraintlayout.widget.ConstraintLayout>

nav_graph :

Tabel 12. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/nav_graph"
7	app:startDestination="@id/homeFragment">
8	
9	<fragment
10	android:id="@+id/homeFragment"
11	android:name="com.example.listxml.HomeFragment"
12	android:label="Home"
13	tools:layout="@layout/fragment_home">
14	<action
15	android:id="@+id/action_home_to_detail"
16	app:destination="@id/detailFragment" />
17	</fragment>
18	
19	<fragment
20	android:id="@+id/detailFragment"
21	android:name="com.example.listxml.DetailFragment"
22	android:label="Detail"
23	tools:layout="@layout/fragment_detail">
24	<argument
25	android:name="title"
26	app:argType="string" />
27	<argument
28	android:name="subtitle"
29	app:argType="string" />
30	<argument
31	android:name="imageResId"
32	app:argType="integer" />
33	</fragment>
	</navigation>

DetailFragment.kt :

Tabel 13. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.navArgs

```

9 import com.example.listxml.databinding.FragmentDetailBinding
10 import timber.log.Timber
11
12 class DetailFragment : Fragment() {
13     private var _binding: FragmentDetailBinding? = null
14     private val binding get() = _binding!!
15     private val args: DetailFragmentArgs by navArgs()
16
17     override fun onCreateView(
18         inflater: LayoutInflater, container: ViewGroup?,
19         savedInstanceState: Bundle?
20     ): View {
21         _binding = FragmentDetailBinding.inflate(inflater,
22 container, false)
23         return binding.root
24     }
25
26     override fun onViewCreated(view: View, savedInstanceState:
27 Bundle?) {
28         Timber.d("DetailFragment: Received item - Title:
29 ${args.title}, Subtitle: ${args.subtitle}, ImageResId:
30 ${args.imageResId}")
31
32         binding.detailTitle.text = args.title
33         binding.detailSubtitle.text = args.subtitle
34         binding.detailImage.setImageResource(args.imageResId)
35     }
36
37     override fun onDestroyView() {
38         super.onDestroyView()
39         _binding = null
40     }
41 }

```

Fragment_detail.xml:

Tabel 14. Source Code Jawaban Soal 1 XML

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <ScrollView
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     android:padding="16dp">
8
9     <LinearLayout
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:orientation="vertical">

```

13	
14	
15	<com.google.android.material.imageview.ShapeableImageView
16	android:id="@+id/detailImage"
17	android:layout_width="match_parent"
18	android:layout_height="200dp"
19	android:scaleType="centerCrop"
20	
21	app:shapeAppearanceOverlay="@style/RoundedImageView" />
22	
23	<TextView
24	android:id="@+id/detailTitle"
25	android:layout_width="wrap_content"
26	android:layout_height="wrap_content"
27	android:textStyle="bold"
28	android:textSize="20sp"
29	android:paddingTop="8dp" />
30	
31	<TextView
32	android:id="@+id/detailSubtitle"
33	android:layout_width="wrap_content"
	android:layout_height="wrap_content" />
	</LinearLayout>
	</ScrollView>

MainActivity.kt:

Tabel 15. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import com.example.listxml.databinding.ActivityMainBinding
6	import timber.log.Timber
7	
8	class MainActivity : AppCompatActivity() {
9	private lateinit var binding: ActivityMainBinding
10	
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	Timber.plant(Timber.DebugTree())
14	binding = ActivityMainBinding.inflate(layoutInflater)
15	setContentView(binding.root)
16	}
17	}

activity_main.xml:

Tabel 16. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.fragment.app.FragmentContainerView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_host_fragment"
6	android:name="androidx.navigation.fragment.NavHostFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	app:navGraph="@navigation/nav_graph"
10	app:defaultNavHost="true" />

ItemViewModel.kt :

Tabel 17. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	import kotlinx.coroutines.launch
8	import timber.log.Timber
9	
10	class ItemViewModel(private val param: String) : ViewModel() {
11	private val _itemList =
12	MutableStateFlow<List<Item>>(emptyList())
13	val itemList: StateFlow<List<Item>> = _itemList
14	
15	private val _selectedItem = MutableStateFlow<Item?>(null)
16	
17	init {
18	viewModelScope.launch {
19	val items = getDummyItems()
20	_itemList.value = items
21	Timber.d("ItemViewModel: \$param - Loaded
22	\${items.size} items")
23	}
24	}
25	
26	
27	fun selectItem(item: Item) {
28	_selectedItem.value = item
29	Timber.d("ItemViewModel: Selected item: \${item.title}")
30	}
31	
32	private fun getDummyItems(): List<Item> {
33	return listOf(
34	Item(R.drawable.lookism, "Lookism", "Author 1",

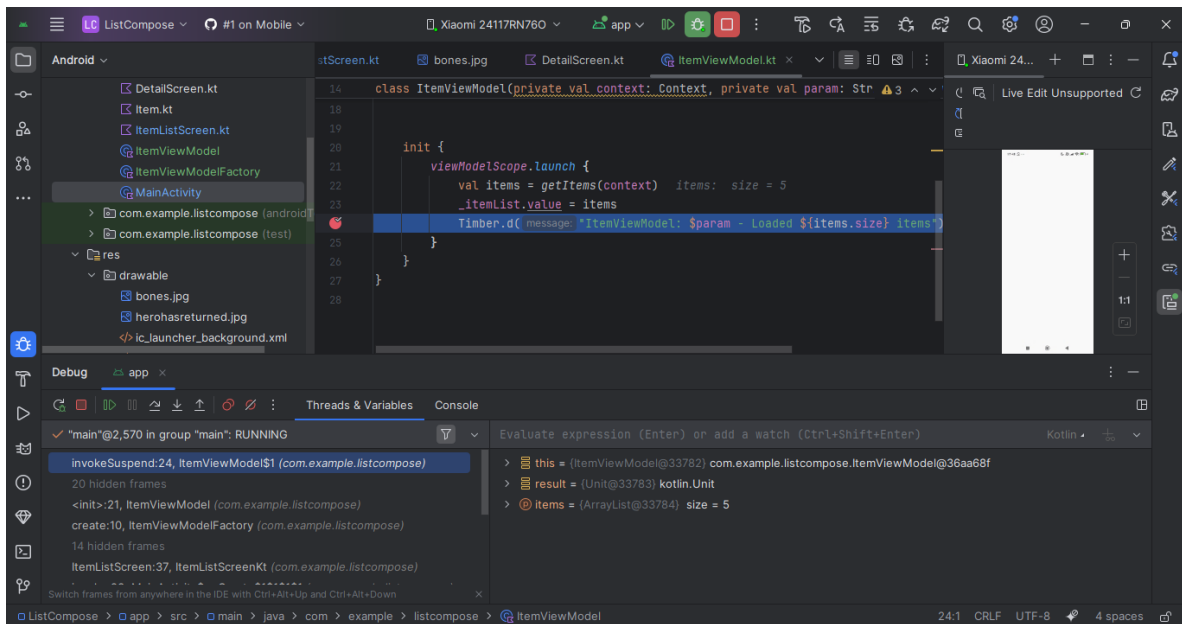
35	"https://lookism.fandom.com/wiki/Lookism"),
36	Item(R.drawable.bones, "Bones", "Author 2",
37	"https://en.namu.wiki/w/%EB%B3%B8%EC%A6%88(%EC%9B%B9%ED%88%B0)"),
38	Item(R.drawable.wee, "Wee", "Author 3",
39	"https://webtoon.fandom.com/id/wiki/WEE!!!"),
	Item(R.drawable.herohasreturned, "Hero Has Returned",
	"Author 4", "https://hero-has-
	returned.fandom.com/wiki/Hero_Has_Returned_Wiki"),
	Item(R.drawable.studygroup, "Study Group", "Author
	5", "https://study-group.fandom.com/wiki/Study_Group_Wiki")
	)
	}
	}

ItemViewModelFactory.kt :

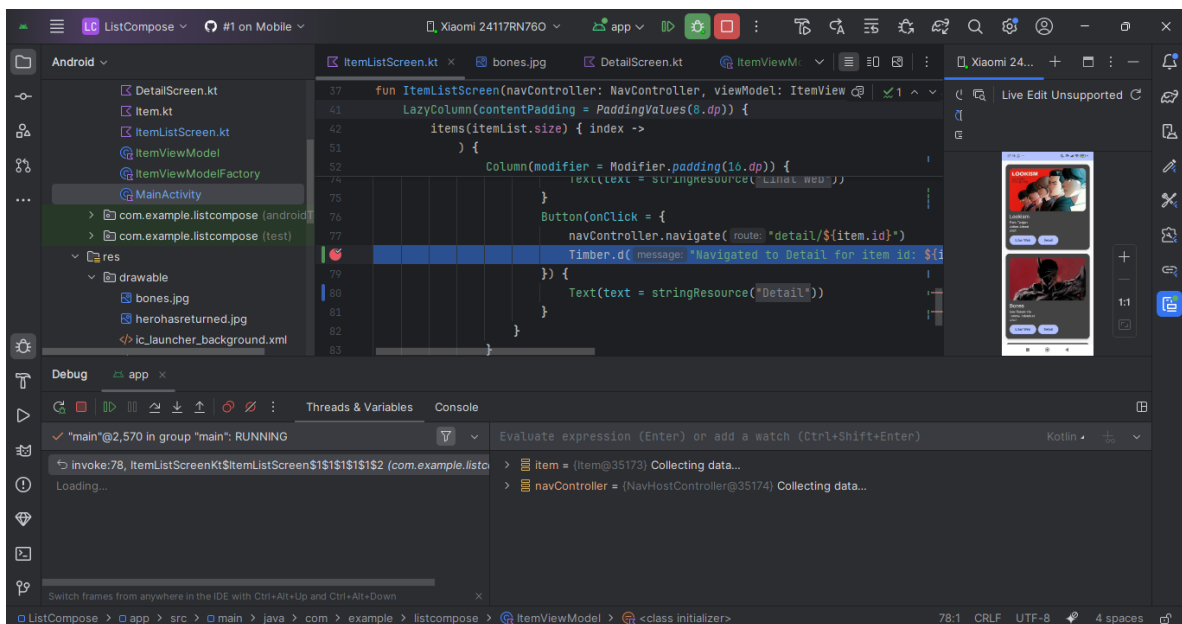
Tabel 18. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class ItemViewModelFactory(
7	private val param: String
8	) : ViewModelProvider.Factory {
9	override fun <T : ViewModel> create(modelClass: Class<T>):
10	T {
11	return ItemViewModel(param) as T
12	}
	}

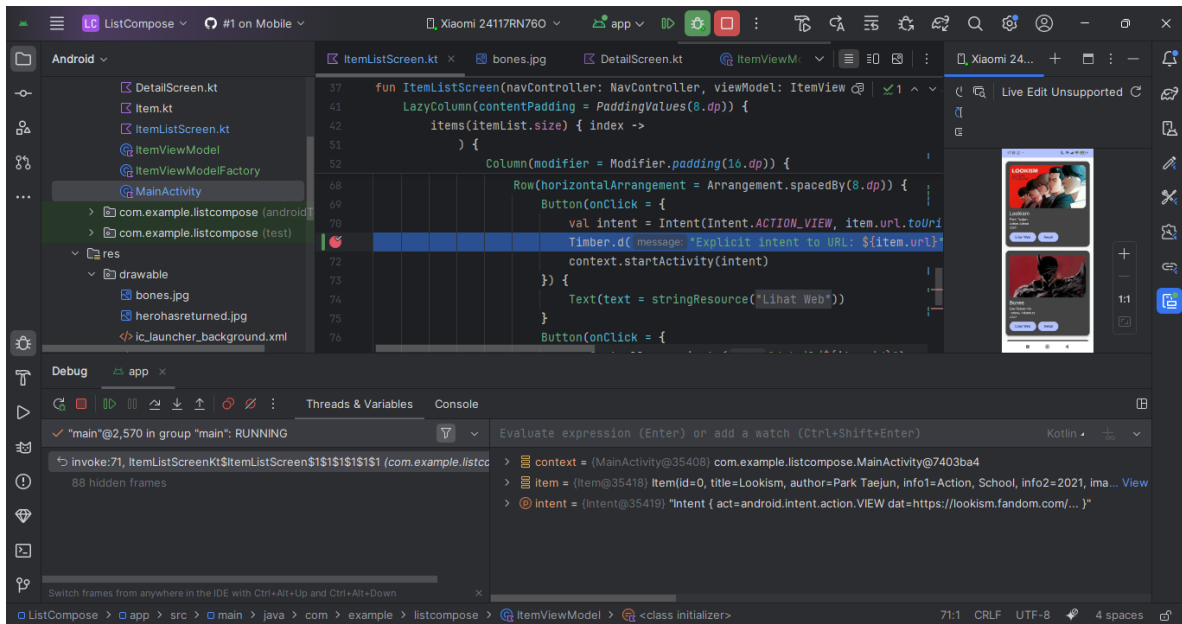
B. Output Program Compose :



Gambar 2. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

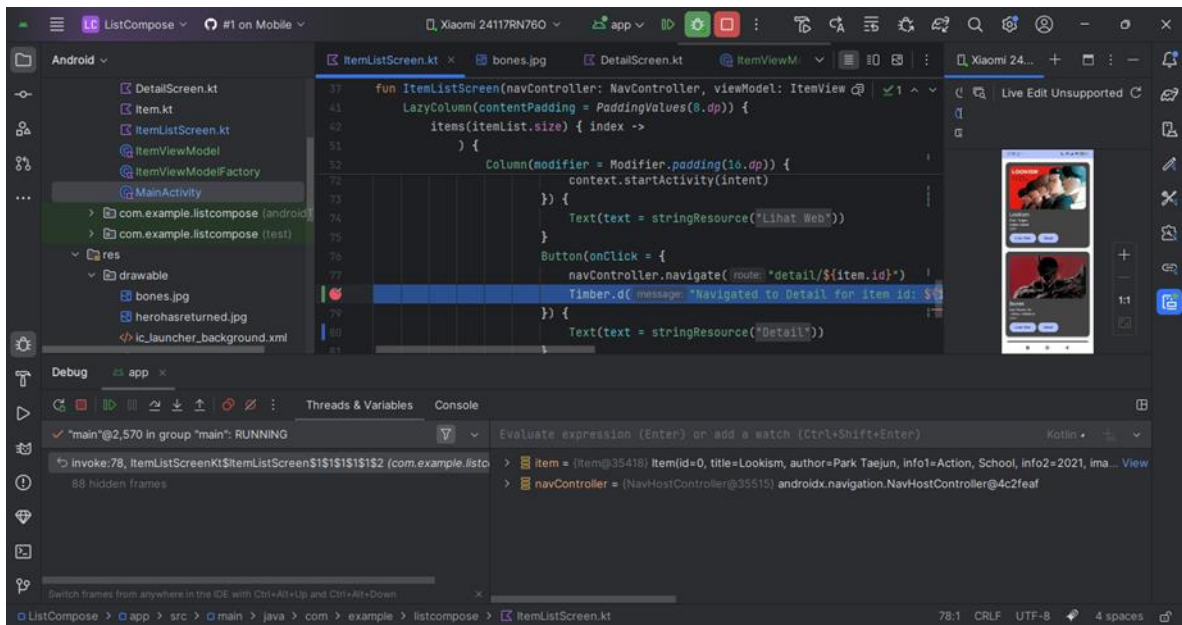


Gambar 3. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

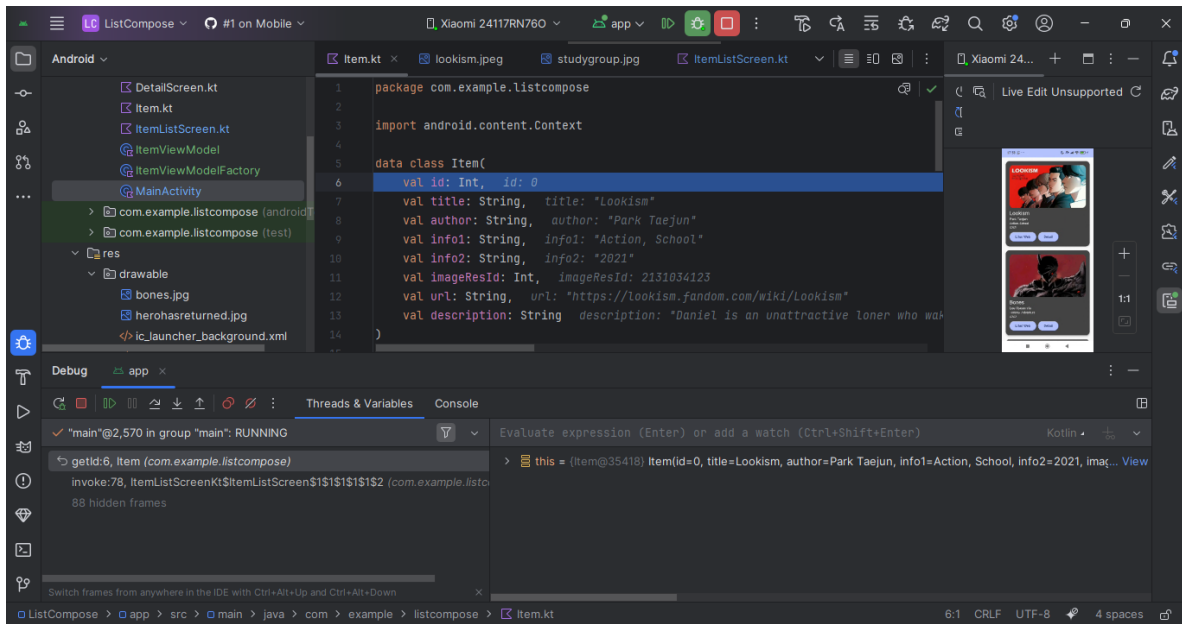


Gambar 4. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Step Into :

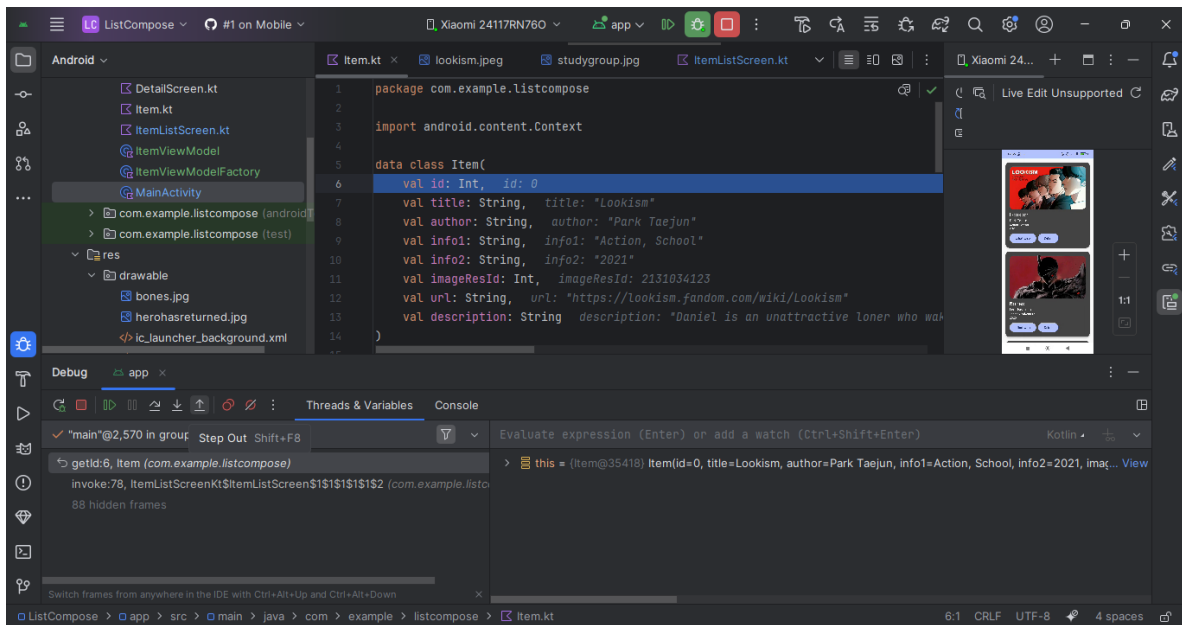


Gambar 5. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

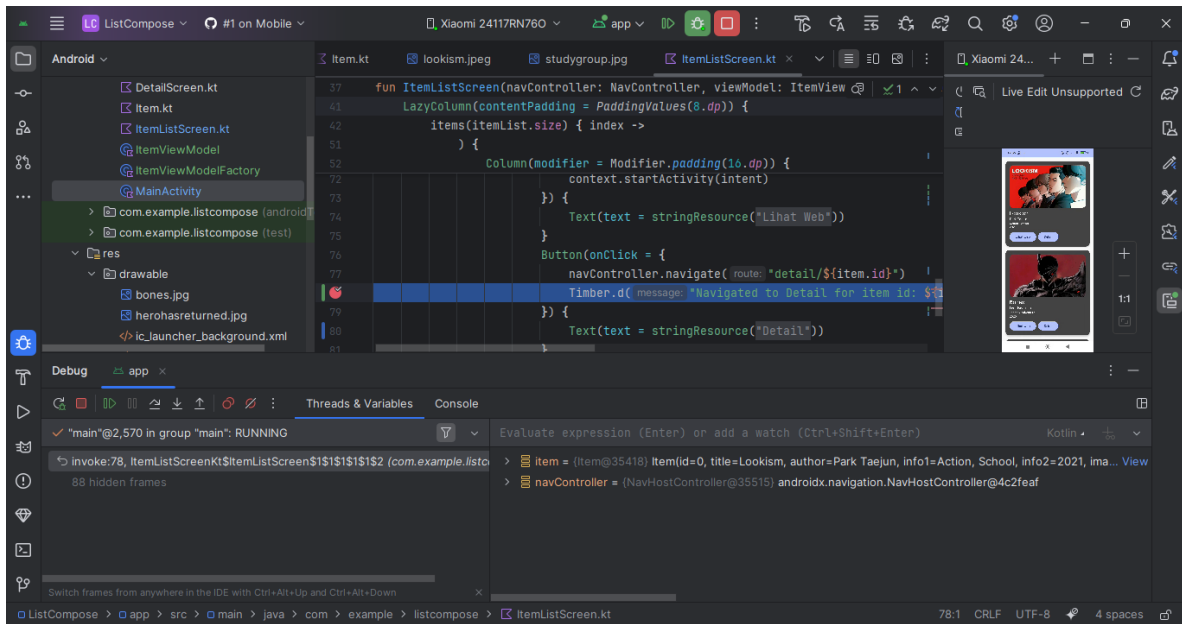


Gambar 6. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Step Out :

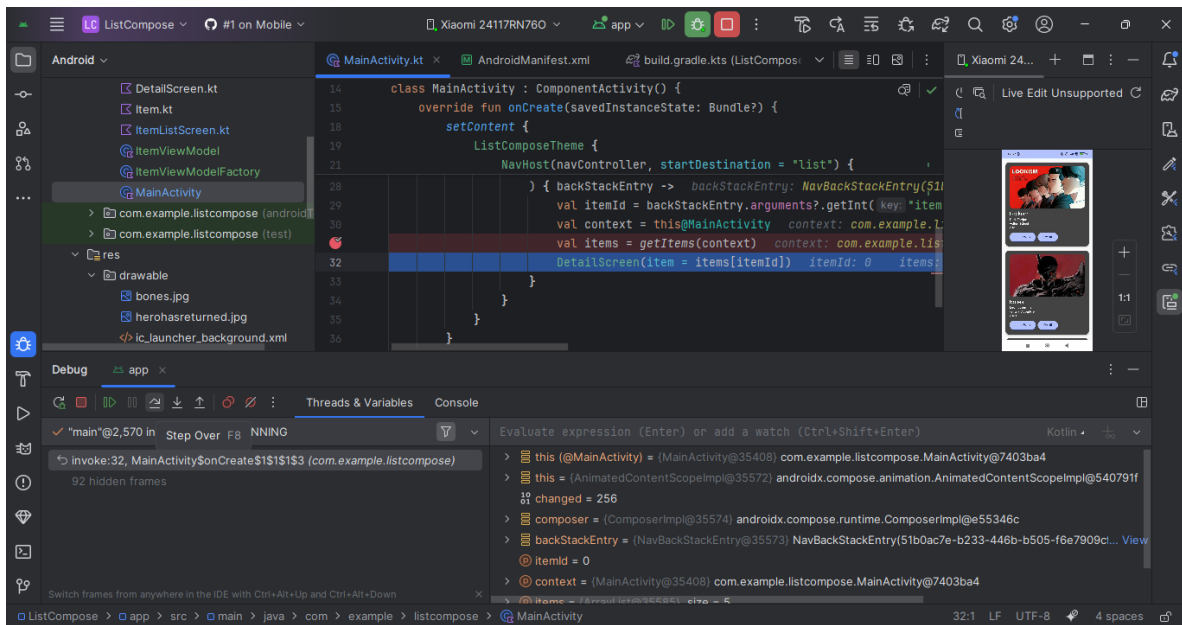


Gambar 7. Screenshot Hasil Jawaban Soal 1 Jetpack Compose



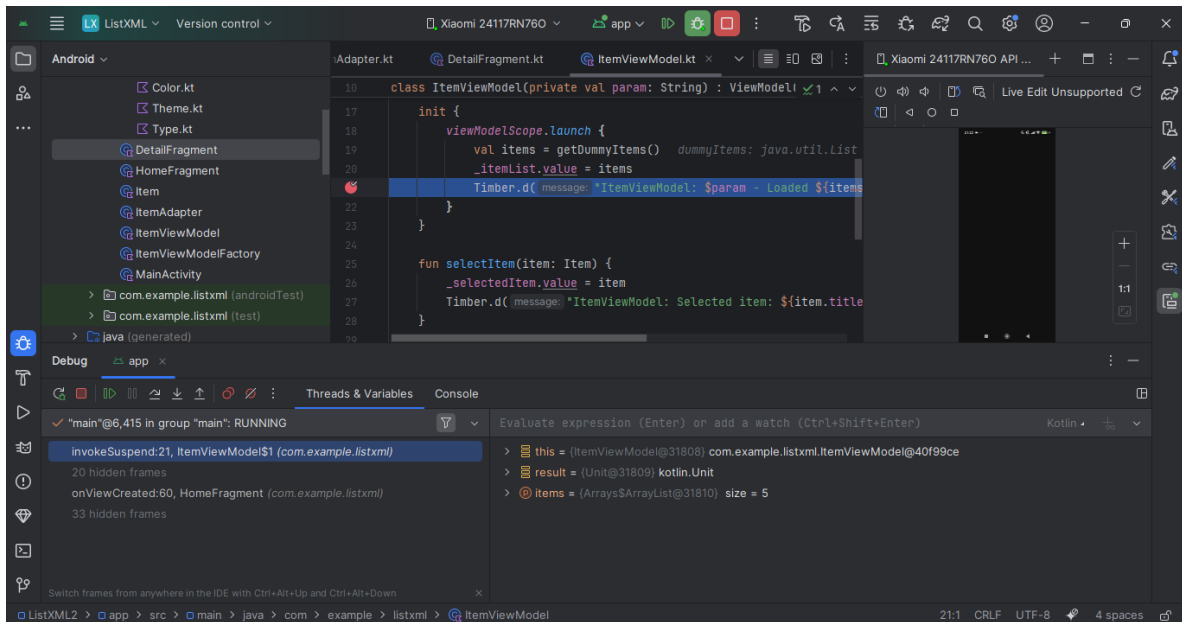
Gambar 8. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Step Over :

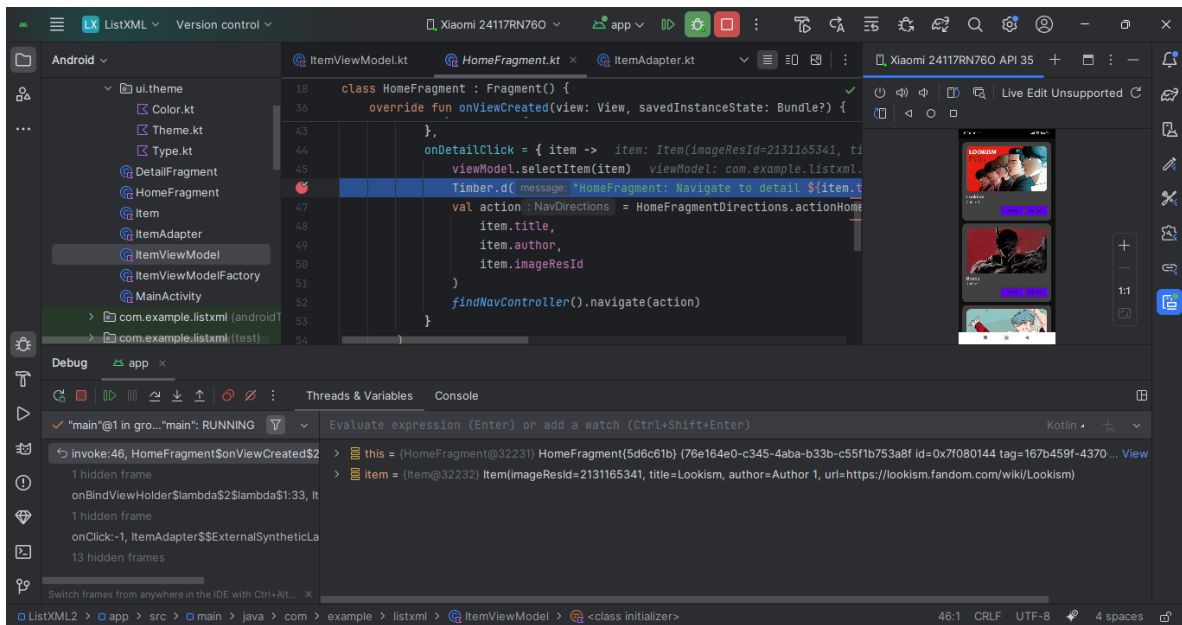


Gambar 9. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

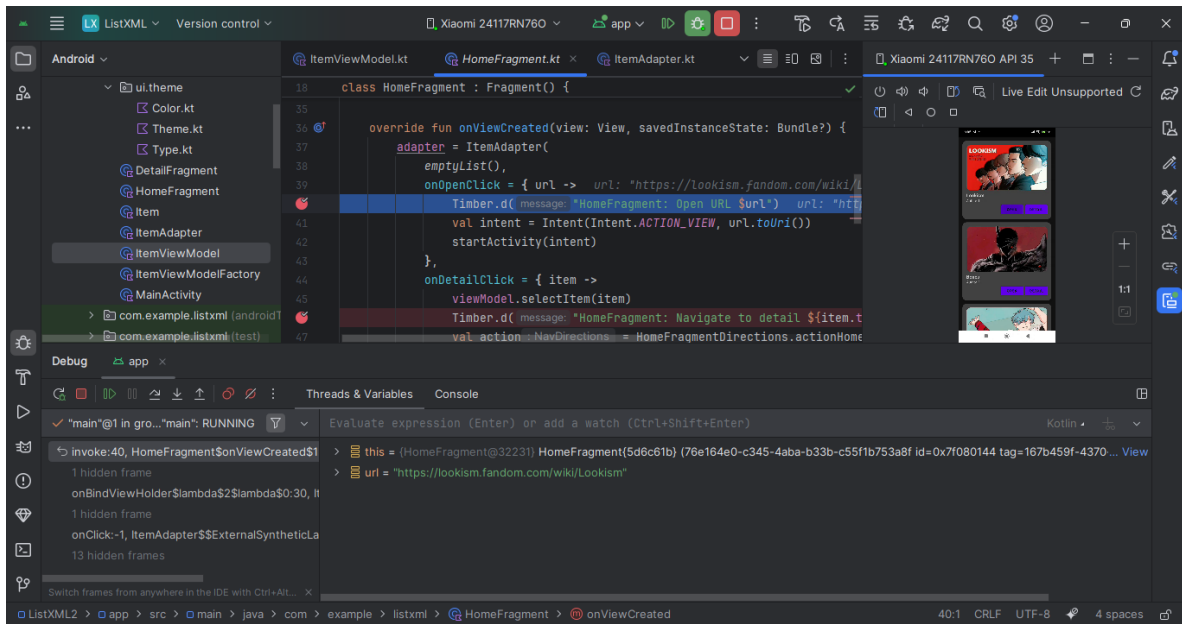
XML :



Gambar 10. Screenshot Hasil Jawaban Soal 1 XML

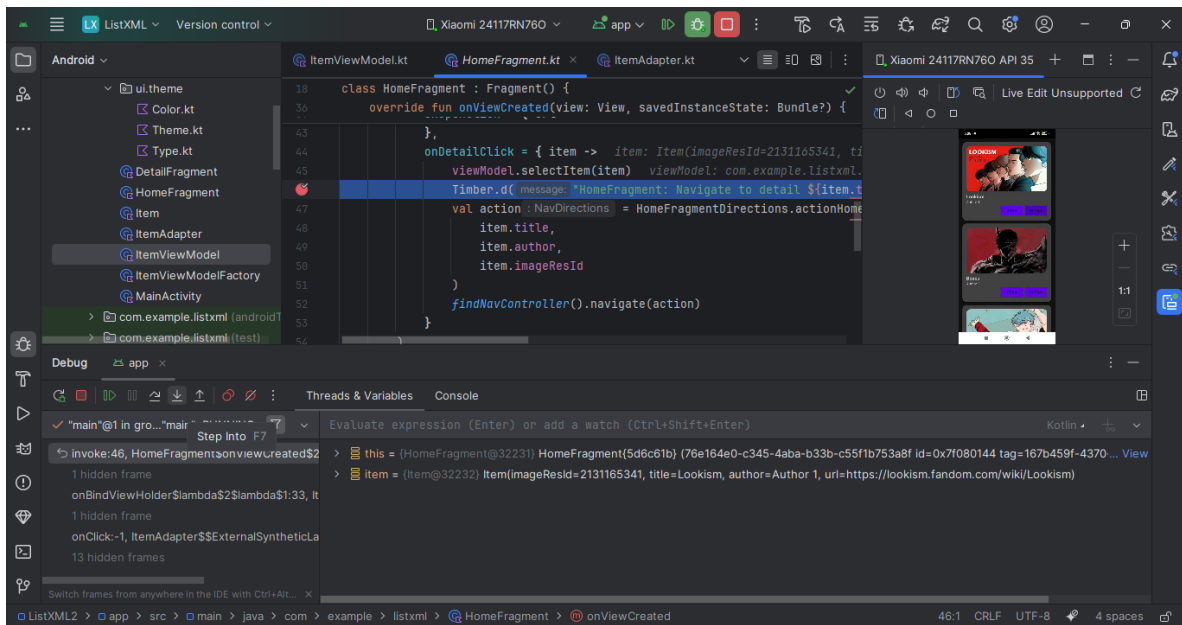


Gambar 11. Screenshot Hasil Jawaban Soal 1 XML

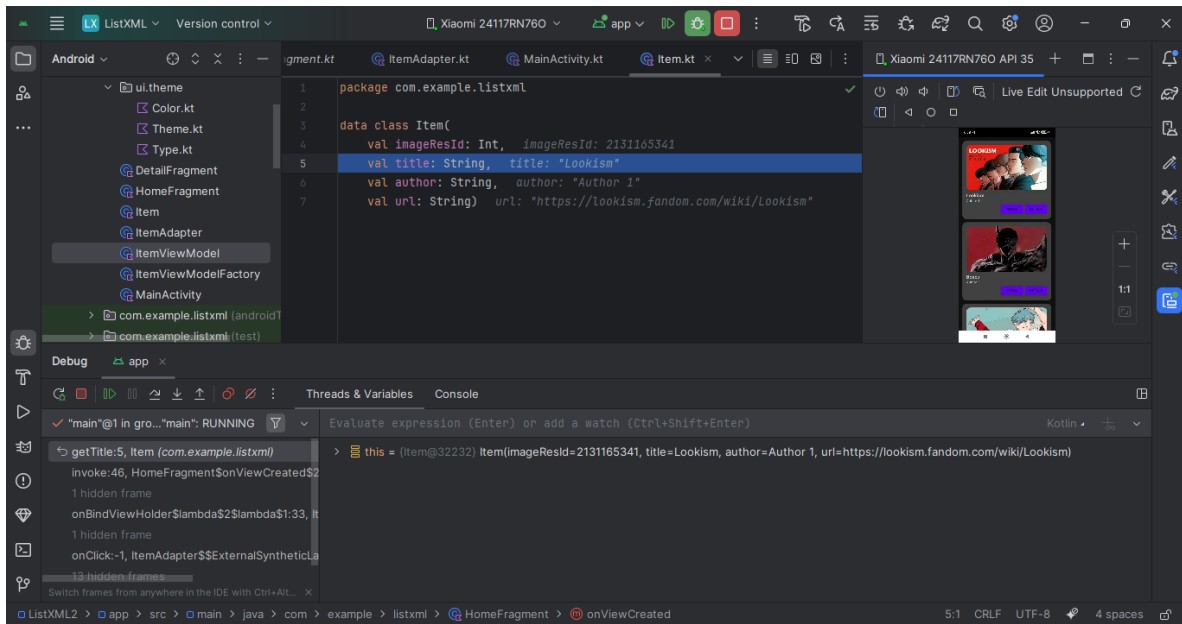


Gambar 12. Screenshot Hasil Jawaban Soal 1 XML

Step Into :

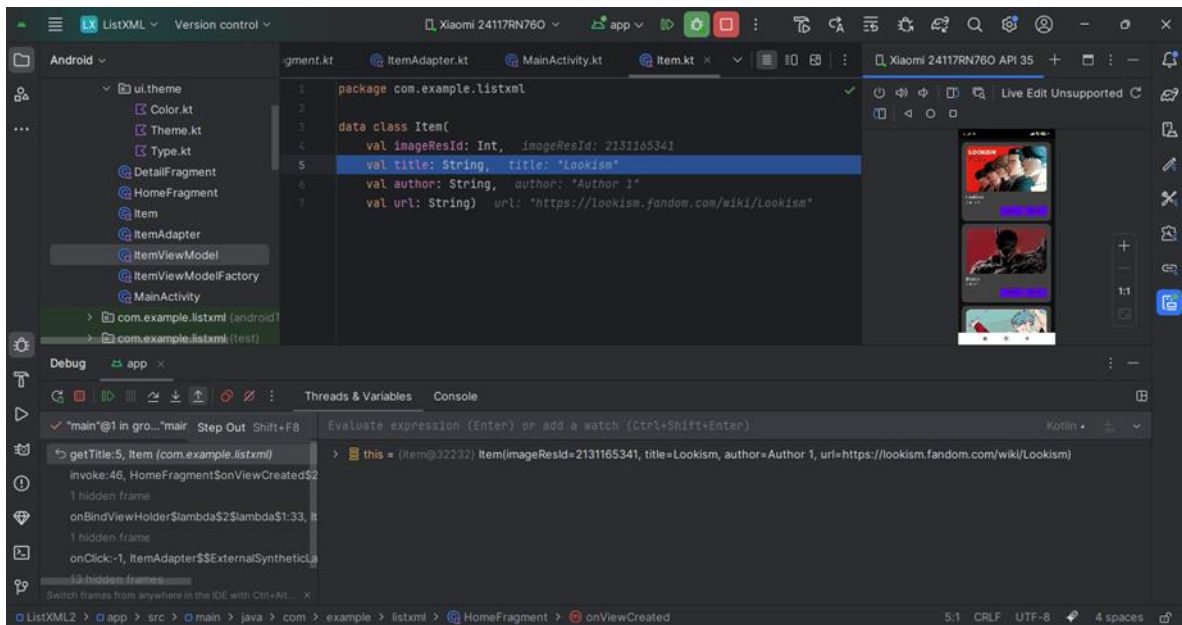


Gambar 13. Screenshot Hasil Jawaban Soal 1 XML

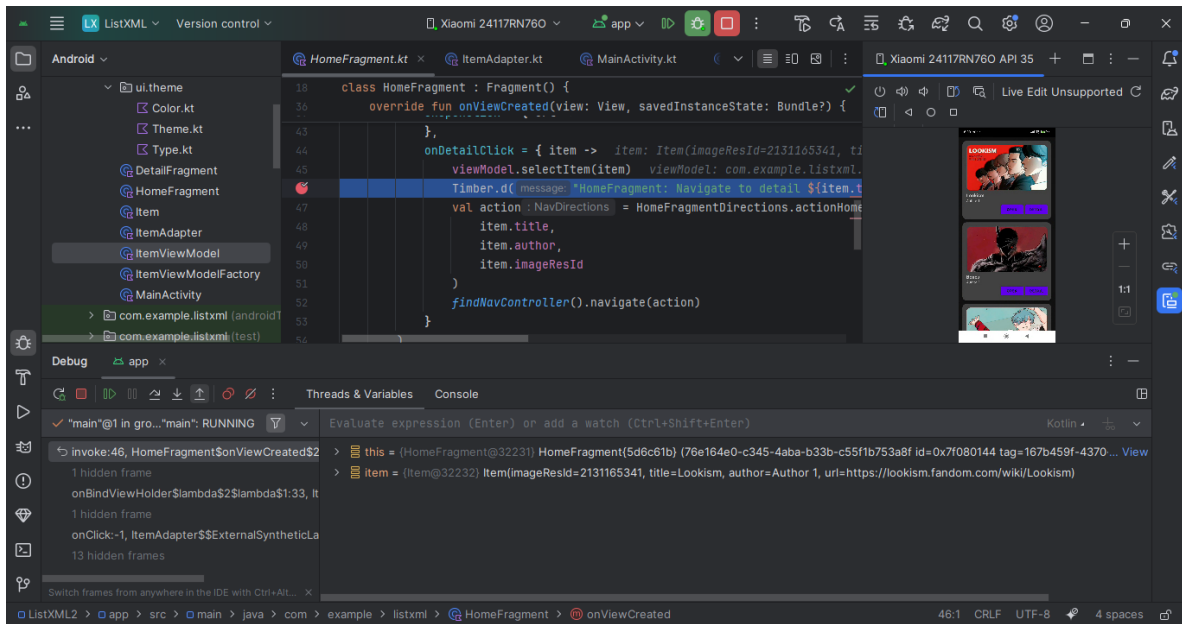


Gambar 14. Screenshot Hasil Jawaban Soal 1 XML

Step Out :

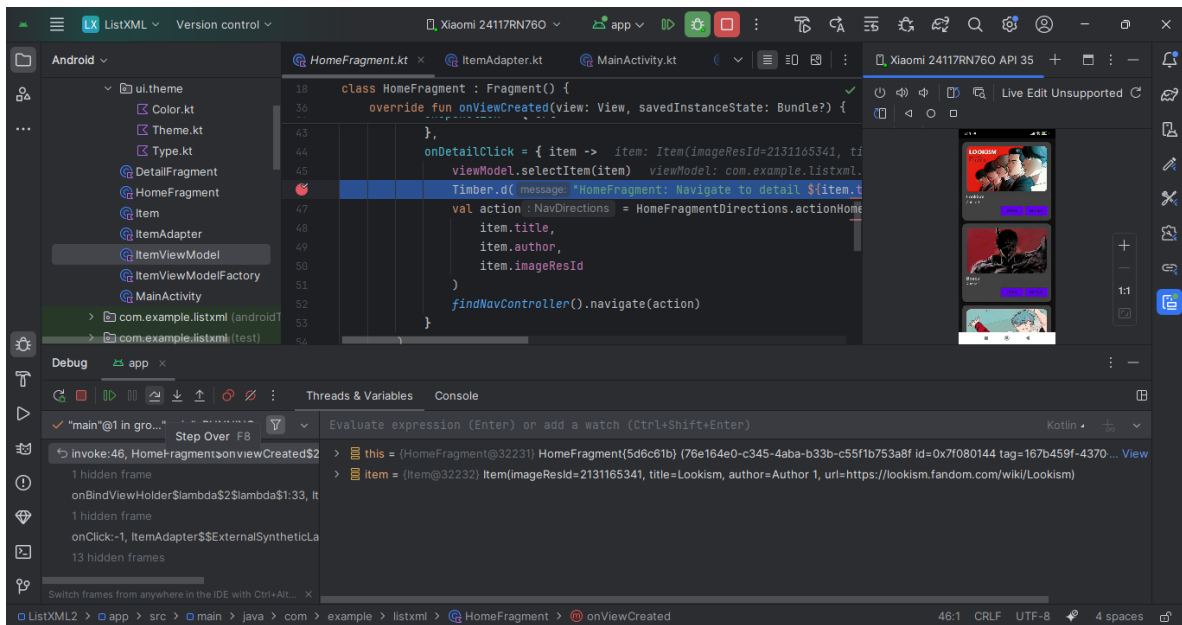


Gambar 15. Screenshot Hasil Jawaban Soal 1 XML

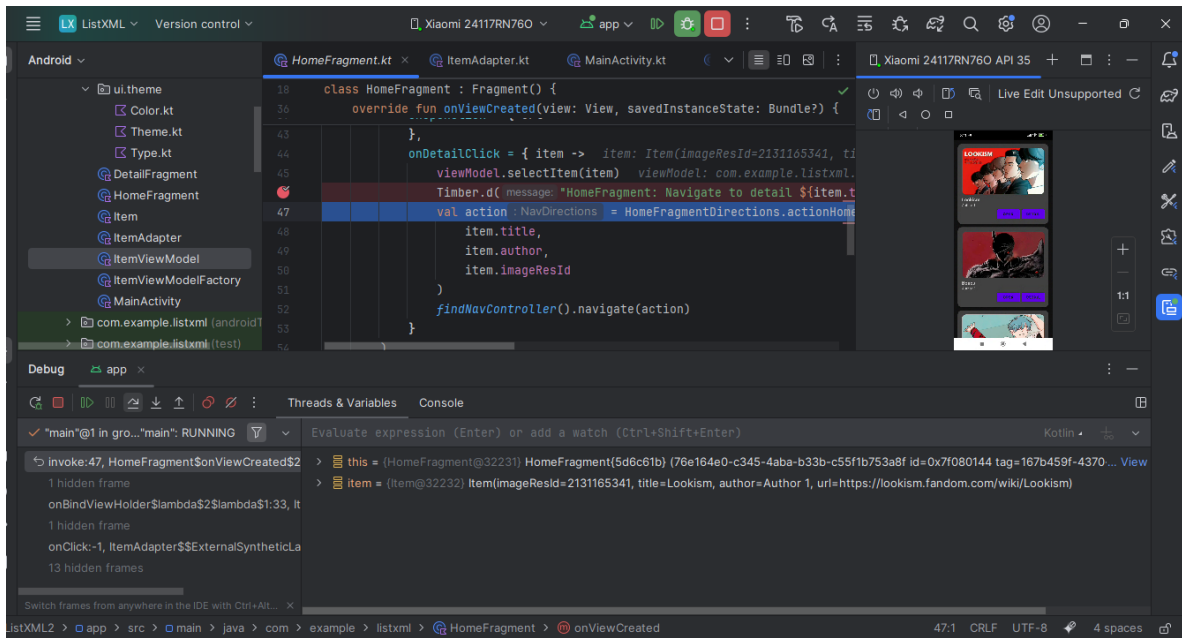


Gambar 16. Screenshot Hasil Jawaban Soal 1 XML

Step Over :



Gambar 17. Screenshot Hasil Jawaban Soal 1 XML



Gambar 18. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

Fungsi debugger :

- Menemukan penyebab **NullPointerException**, **crash**, atau hasil yang tidak sesuai.
- Melihat data yang masuk ke ViewModel, RecyclerView, atau Intent.
- Mengecek perubahan nilai StateFlow, LiveData, atau variabel lainnya.

Cara menggunakan debugger :

1. Pasang Breakpoint

- Klik di samping nomor baris kode (lingkaran merah akan muncul).
- Misalnya: `Timber.d("Data: $data")` diberi breakpoint.

2. Jalankan Aplikasi dalam Mode Debug

- Klik tombol "Debug App" di Android Studio.
- Aplikasi akan berjalan dan berhenti di baris yang diberi breakpoint.

3. Gunakan Panel Debug

- Lihat nilai variabel di panel "Debugger".
- Bisa menambahkan Watch, melihat Stack Trace, dan mengontrol eksekusi.

Fitur Step Into, Step Over, dan Step Out :

Step Into : Masuk ke dalam fungsi/metode yang sedang dipanggil.

Step Over : Loncat ke baris berikutnya tanpa masuk ke dalam fungsi/metode.

Step Out : Keluar dari fungsi saat ini dan kembali ke fungsi pemanggil.

Jetpack Compose :

MainActivity.kt:

Dalam kode MainActivity, digunakan library Timber untuk keperluan logging selama proses pengembangan aplikasi. Logging sangat penting untuk memonitor alur program, memeriksa data yang sedang diproses, serta melacak bug atau error.

Line 17, digunakan untuk mengaktifkan Timber agar bisa menampilkan log saat aplikasi dijalankan dalam mode debug. DebugTree() merupakan implementasi Timber yang cocok digunakan selama proses pengembangan karena akan mencetak log ke Logcat.

Line 31, dapat dipasang **breakpoint** untuk menghentikan sementara eksekusi program saat sedang dijalankan dalam mode **debug**. Tujuannya adalah untuk:

- Melihat isi data yang dikembalikan oleh fungsi `getItems(context)`.
- Memastikan bahwa data items berisi list yang benar sebelum diteruskan ke DetailScreen.
- Memantau apakah proses pemanggilan data dari resource (judul, deskripsi, gambar) berhasil.

Setelah breakpoint dipasang dan aplikasi dijalankan dalam **mode debug**, kita dapat menggunakan fitur **Step Over (F8)** untuk melanjutkan ke baris berikutnya tanpa masuk ke dalam fungsi, atau **Step Into (F7)** untuk menelusuri bagaimana `getItems()` membentuk daftar Item.

ItemListScreen.kt :

Dalam ItemListScreen.kt, library Timber digunakan untuk melakukan logging aktivitas pengguna ketika tombol ditekan. Logging ini sangat membantu saat proses debugging karena memberikan informasi langsung ke Logcat Android Studio mengenai tindakan yang terjadi di aplikasi.

Line 70, mencatat pesan log ketika pengguna menekan **tombol "Website"** (explicit intent). Breakpoint bisa dipasang di baris ini untuk:

- Memastikan bahwa URL dari item yang dipilih benar dan sesuai harapan.
- Memverifikasi bahwa aksi Intent akan dilakukan ke alamat yang valid.

- Melihat kapan dan bagaimana fungsi startActivity dipicu untuk membuka halaman web.

Ini penting untuk memastikan bahwa **intent eksplisit** benar-benar membawa pengguna ke halaman manhua yang diinginkan.

Line 77, Logging ini dipicu ketika pengguna menekan tombol "**Detail**", yang akan mengarahkan ke halaman DetailScreen. Breakpoint di sini berguna untuk:

- Memeriksa nilai item.id yang dipilih dan akan dikirim melalui NavController.
- Menjamin bahwa navigasi dilakukan ke halaman detail dengan data yang benar.
- Memastikan bahwa event klik benar-benar ditangkap dan dijalankan oleh sistem navigasi.

ItemViewModel.kt :

Dalam file ItemViewModel.kt, penggunaan Timber dilakukan pada bagian inisialisasi ViewModel, tepatnya setelah data items berhasil dimuat melalui fungsi `getItems(context)`.

Line 20, Baris ini berfungsi untuk mencatat **jumlah data item** yang berhasil dimuat dari sumber data (`getItems(context)`) saat ViewModel pertama kali dijalankan. Logging ini membantu **developer mengetahui apakah data berhasil diambil** dan berapa banyak jumlahnya.

- Nilai `$param` berguna untuk memberi label log yang berbeda jika ViewModel digunakan di beberapa tempat dengan parameter yang berbeda.
- `${items.size}` menampilkan jumlah item yang di-load ke dalam aplikasi.

XML :

HomeFragment.kt :

Di dalam HomeFragment, Timber digunakan untuk mencatat proses penting yang terjadi saat pengguna berinteraksi dengan daftar item. Logging ini membantu proses debugging dan pelacakan alur kerja aplikasi secara real-time melalui **Logcat** Android Studio.

Line 40, Baris ini digunakan untuk mencatat log saat pengguna menekan tombol "Open" pada salah satu item di dalam daftar. Log ini berfungsi untuk memastikan bahwa URL yang akan dibuka melalui intent eksplisit sudah benar dan proses intent telah dipicu.

Line 46, Logging ini mencatat saat pengguna menekan tombol "Detail" pada item. Ini menunjukkan bahwa data item telah berhasil dipilih oleh ViewModel, dan proses navigasi ke halaman Detail telah dimulai, beserta judul item yang sedang diproses.

Line 62, Baris ini mencatat jumlah data yang diterima dari ViewModel dan ditampilkan ke dalam RecyclerView. Logging ini penting untuk memastikan data benar-benar terkirim dan diperbarui ke adapter.

DetailFragment.kt :

Pada DetailFragment.kt, Timber digunakan sebagai alat pencatat log (*debug logging*) untuk menampilkan informasi data yang diterima dari HomeFragment melalui Safe Args.

Penggunaan Timber ini sangat penting untuk membantu praktikan memverifikasi apakah data yang dikirim saat navigasi benar-benar sesuai.

Line 26, Baris log ini mencatat informasi yang diterima oleh DetailFragment setelah navigasi dilakukan dari HomeFragment. Log menampilkan data judul (title), penulis/subjudul (subtitle), dan ID gambar (imageResId) dari item yang dipilih. Tujuannya adalah memastikan bahwa semua data diteruskan dengan benar melalui argument navArgs.

MainActivity.kt :

Line 13, Baris ini berada di dalam fungsi onCreate dan digunakan untuk menginisialisasi Timber, yaitu sebuah library logging yang lebih fleksibel dan rapi dibandingkan Log.d bawaan Android.

ItemViewModel.kt :

Pada file ini, Timber digunakan untuk mencatat log debug yang berkaitan dengan dua hal penting dalam ViewModel:

1. Proses pemanggilan dan pemuatan data dummy item.
2. Proses pemilihan item oleh pengguna.

Line 21, Baris log ini dijalankan setelah daftar item dummy berhasil dimuat ke dalam _itemList. Log mencatat jumlah item yang dimuat dan menampilkan nilai param yang dikirim ke ItemViewModel.

Line 27, Log ini dicetak ketika pengguna memilih salah satu item dari daftar. Log menampilkan judul item yang dipilih, sehingga bisa dipastikan bahwa proses pemilihan berjalan dengan benar.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul4>

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application class dalam arsitektur aplikasi Android adalah kelas dasar yang merepresentasikan seluruh aplikasi secara global dan dibuat saat aplikasi dijalankan. Fungsinya adalah untuk menginisialisasi komponen-komponen yang bersifat global dan hanya perlu dijalankan sekali selama aplikasi aktif, seperti setup library logging, dependency injection, atau database. Selain itu, Application class juga dapat menyimpan data atau state yang harus tersedia selama aplikasi berjalan. Untuk menggunakannya, kita membuat subclass dari Application dan mendaftarkannya di file AndroidManifest.xml sehingga sistem Android mengetahui kelas mana yang digunakan sebagai Application class. Application class membantu mengelola konfigurasi dan data global agar aplikasi berjalan dengan lebih terstruktur dan efisien.